

Product Requirements Document (PRD)

App Cleaner — Uninstall Apps · Android V1

App Name	App Cleaner: Uninstall Apps
Version	1.1
Date	September 28, 2026
Status	Ready for Development (V1)
Author	Product — Jedy Apps
Platform	Android Native (Kotlin)

Scope in one line. A free batch uninstaller with an honest storage overview, plus a weekly subscription (3-day free trial) that unlocks the Unused Apps finder, the per-app storage breakdown, cleanup reminders and an ad-free experience. No connection gate, no account, no backend.

0. Objective & Core Logic

Ship the fastest way on Android to go from “my storage is full” to “those apps are gone” — select a handful of apps once, confirm them in a row, and see exactly how much space came back. Android buries uninstalling under Settings → Apps → app → Uninstall, one app at a time; on a phone with 120 apps that is the reason people never clean up. The product target is a median of under 60 seconds from app open to the first completed batch.

The whole app is three verbs. **Find -> Select -> Remove**. Every screen in this document serves one of them. Free users get Find by name, size and date, and unlimited Remove. The subscription sells a better Find — which apps you have actually stopped using, and which ones really eat the storage — because that is the question a plain app list can't answer.

The five architectural decisions everything else follows from:

- **Android uninstalls, we queue.** The app never removes a package itself. It hands each selected package to `PackageInstaller.uninstall()` and Android shows its own confirmation dialog, one app after another. Every call returns a real per-app result (success, cancelled, blocked) through an `IntentSender`, so the result screen reports what actually happened rather than what we hoped happened. No Accessibility auto-tapping, no root — nothing that Play policy can pull the listing over.
- **Measure before you delete.** Once a package is uninstalled, its icon, label and size are gone from `PackageManager` for good. So the app snapshots all three into Room *before* each confirmation dialog opens. The “Freed 2.4 GB” number and the entire Uninstall History are built from that snapshot — skipping it is why competitor history screens show blank icons and “0 B”.
- **Show the count before the price.** Usage Access is requested for free, before any paywall. The Unused and Large tabs then compute the real answer on-device and show it blurred: “14 apps you haven't opened in 60 days · 3.4 GB”. The number is the pitch. A paywall that says “find unused apps” converts on curiosity; one that says *you have 14* converts on evidence.
- **The app inventory never leaves the phone.** The list of apps a person has installed is personal and sensitive data under Google Play's User Data policy. No network call, analytics event or crash log may carry a package name or app label — events carry counts and byte buckets only (Section 9).
- **An honest cleaner.** “App Cleaner” means cleaning apps off the phone. Since Android 6, a normal app cannot clear another app's cache or “junk” without root, so V1 never claims to. The storage overview shows cache sizes truthfully and routes “clear cache” to the system App info page. Fake boost/junk claims are the #1 reason cleaner apps get suspended under Play's Deceptive Behavior policy — we sidestep the entire category risk by

not making them.

1. Configuration, ASO & Design System

- **App Name:** App Cleaner: Uninstall Apps (27 characters). “App Cleaner” is the brand, “Uninstall Apps” is the highest-volume search phrase in the category and mirrors how the leaders are titled (“Uninstaller”, “Delete apps - Easy Uninstall”, “Uninstall Apps – App Remover”). The bare name “App Cleaner” is generic and already crowded (e.g. “Uninstaller - My App Cleaner”, `uninstaller.apps.uninstall.free`), so the keyword suffix is what makes the title rank and distinct.
- **Reference App(s):** Delete apps - Easy Uninstall (`com.mobique.deleteapps`) — 1M+ installs, 4.4 (33.6K), the primary reference for its one-screen select-and-remove flow; Uninstaller (`com.splendapps.shark`) — 10M+, 4.8 (256K), for sort modes and the storage bar; Easy Uninstaller (`mobi.infolife.uninstaller`) — 10M+, 4.5 (181K), for uninstall history and usage tracking; AppDrop (`com.bulk.uninstall`) — 50K+, 4.2, as the Accessibility auto-uninstall approach we deliberately do not take. Full findings in Appendix A.
- **Design Kit / Figma Reference:** Custom Clean UI, light and dark, “Clean Teal” primary.
- **Bundle ID:** `com.jedy.appcleaner.uninstaller`
- **Platform:** Android Native (Kotlin, Jetpack Compose). Min SDK 26 — `StorageStatsManager` (per-app sizes) and `StorageManager.getAllocatableBytes` both start at API 26, and Android 7 is under 2% of active devices.
- **Supported Languages:** English, Spanish, French, German, Mandarin, Hindi, Arabic, Hebrew — the `InkSign` set, reusing its `AppLanguage / LocaleController` module unchanged.
 - Layout Rule: Strict Left-To-Right (LTR) enforced globally, with dynamic switching to Right-To-Left (RTL) ONLY when Arabic or Hebrew is selected during onboarding or in device settings.
- **Target Audience:** Someone standing in front of the “Storage almost full” notification, trying to take a video or install an update right now. Also the person handing down an old phone, and the parent clearing the eleven games a child installed over a weekend. The unifying trait is not a demographic — it is a phone with far more apps than its owner remembers installing, and a need to get space back in the next two minutes.
- **UI/UX Specifications:**
 - Theme & Palette: Follows the system theme (light/dark), overridable in Settings. Light: White background (`#FFFFFF`), surface cards `#F5F7F8`, primary text `#111827`, secondary text `#6B7280`. Dark: background `#0F1417`, surface `#182125`, primary text `#F3F4F6`, secondary text `#9CA3AF`. “Clean Teal” primary accent (`#0F9D8A` light / `#2DD4BF` dark) for CTAs, checkboxes, the selection bar, tabs and the storage bar. “Remove Red” (`#E5484D`) is reserved exclusively for the Uninstall action and destructive confirmations — it must never be used for a generic button, so that red always means “this removes something”. “Premium Gold” (`#F5B301`) is used only for the crown badge on premium tabs and the paywall.
 - Design Integrity: Flat, list-first. Rows are 72dp with a 40dp app icon, label, one line of metadata (“248 MB · Installed Mar 2024”) and a trailing checkbox. Cards with subtle grey borders instead of heavy shadows, 12dp–16dp corner radius, Google Material Symbols (Rounded). Motion is limited to state changes: the selection bar sliding up, and the freed-space counter counting up on the Result screen — nothing decorative.

2. Mandatory V1 Infrastructure & Technical Policies

- **Android API Target:** Must target Android 16 (API level 36) or higher.
- **Billing & Purchases:** Google Play Billing Library (MUST be v8.0.0+, v9 recommended) alongside RevenueCat for purchase state management. V1 sells exactly one product: a **weekly auto-renewing subscription with a 3-day free trial** (Play base plan + free-trial offer), mapped to a single RevenueCat entitlement `premium`. No yearly or lifetime plan in V1 (Section 10).

- **Analytics & Monitoring:** Full integration of Firebase Analytics and Crashlytics — subject to the inventory rule below: no package names or app labels in any event parameter, custom key or log line.
- **Smart Notifications System:** Local notifications via WorkManager, tailored for retention. Premium: the cleanup reminder (Feature 4). Free: at most one generic nudge per 30 days (“Time for a cleanup? You have 126 apps installed”) that needs no usage data.
- **User Privacy & Consent (EU & US):** Google UMP SDK integration for GDPR (EU) and US state privacy regulations.
- **Connectivity:** No connection gate (a deliberate difference from InkSign v1.1). Uninstalling is local work and the revenue is the subscription, not ad impressions; a wall in front of a two-minute cleanup task would buy almost no ad revenue and a lot of one-star reviews. Ads, consent and the paywall price load opportunistically when a connection exists.
- **On-Device Inventory Policy:** No account, no login, no server. The installed-app list, usage timestamps and storage numbers are read, computed and stored on the device only. The Play Data safety form declares “App activity → Installed apps” and “App activity → App interactions” as **processed on-device, not collected**. This is a hard architectural rule and the app's primary trust claim.
- **Restricted Permissions & Play Declarations:**
 - `QUERY_ALL_PACKAGES` — required to list every installed app. File the Play Console permission declaration under the core-purpose use case (app management / uninstaller). A launcher-intent `<queries>` fallback is not acceptable: it silently hides keyboards, widget packs and launcher-less apps, which are exactly the forgotten ones.
 - `REQUEST_DELETE_PACKAGES` — required for `PackageInstaller.uninstall()`. Normal permission, no runtime prompt.
 - `PACKAGE_USAGE_STATS` — special app access (Usage Access), granted by the user in Settings. Preceded by an in-app prominent-disclosure screen (Screen 11) as Play's User Data policy requires.
 - `POST_NOTIFICATIONS` — runtime prompt, requested at the end of onboarding.
 - No `BIND_ACCESSIBILITY_SERVICE`, no `MANAGE_EXTERNAL_STORAGE`, no root paths.
- **Engines & Libraries:** `PackageManager` + `PackageInstaller` for inventory and removal; `UsageStatsManager` for last-used data; `StorageStatsManager` + `StorageManager` for sizes; Room for the cached inventory, size/icon snapshots, the uninstall queue and history; WorkManager for reminders; Coil with a custom `PackageManager` icon fetcher for list icons; Hilt for dependency injection; Jetpack Compose + Material 3 for UI.
- **Performance Budget (acceptance criteria):** cold start to a populated list under 1.0s on a mid-range device (served from the Room inventory cache, refreshed in the background); full inventory refresh of 300 apps under 2.0s; scrolling at 60fps with icons; next confirmation dialog shown within 300ms of the previous result.

Ads SDKs and Subscription models are characterized in separate, dedicated PRDs.

3. Onboarding Experience & Specification

- **Onboarding Type:** Language Selector -> 3-Slide Carousel -> Paywall (dismissible).
- **Visual Layout & Elements:**
 - Initial Screen: Full-screen language selector, defaulting to device locale. If Arabic or Hebrew is selected, the UI instantly flips to RTL.
 - Hero Illustrations: Flat teal-on-white vector graphics (dark variants in teal-on-slate). Slide 1: a phone with a grid of app icons, several ticked. Slide 2: a storage bar shrinking from red-full to teal. Slide 3: a calendar with dusty, faded app icons.
 - Headline & Subtext: Slide 1 “Remove many apps at once” / “Pick them, confirm, done — no digging through Settings.” Slide 2 “See what's eating your storage” / “Every app's size, sorted biggest first.” Slide 3 “Find the

apps you forgot” / “We’ll show you what you haven’t opened in months.”

- Controls & CTAs: “Next” button (bottom end), “Skip” (top end), and a large teal “Get Started” CTA on the final slide.
- **Permission Requests & Sequence:**
 - Step 1: Google UMP GDPR/US consent dialog, on initial launch after the language selector, blocking until resolved. With no connection, UMP cannot load: skip it for this session, treat consent as not granted (non-personalized ads only), and retry on the next cold start.
 - Step 2: Push notification permission prompt, triggered on tapping “Get Started” on the last slide.
 - Step 3: Onboarding Paywall (Screen 3), dismissible with a visible close (X) from the first frame.
 - Step 4: Usage Access, requested contextually only — the first time the user opens the Unused or Large tab, or taps “See full breakdown” on the storage card — always preceded by the disclosure screen (Screen 11). Never during onboarding: sending a first-time user into a system Settings page before they have seen their own app list is the single biggest abandonment point in this category.
- **User Logic & Edge Cases:**
 - Tapping “Skip” immediately writes `onboarding_complete` and still shows the paywall once; closing it lands on Home. The paywall is shown at most once in onboarding — never again on cold start.
 - Swipe gestures and button taps must both be supported for carousel navigation.
 - The inventory scan starts in the background on the language screen, so Home is populated the moment onboarding ends — no empty list with a spinner as the first real screen.
 - Cold start from the reminder notification (premium) on a device where onboarding somehow never completed (data cleared): run the language selector only, then go straight to the Unused tab.

4. Screen Architecture & Screen-by-Screen UI Breakdown

- **Screen 1: Splash Screen** — `SplashScreen` API, teal app icon on the theme background, held only until the Room inventory cache is read (target under 400ms).
- **Screen 2: Onboarding Flow** (detailed in Section 3).
- **Screen 3: Paywall** — Close (X) top-start, always visible. Headline “Find the apps you forgot about”. Four benefit rows with icons: Unused apps finder · Storage used by every app · Cleanup reminders · No ads. When opened from a locked tab, the headline is replaced with the real finding (“14 apps unused for 60+ days · 3.4 GB”). One plan card: “3 days free, then <price>/week”. Large teal CTA “Start free trial”. Beneath it, the Play-required terms line: “Auto-renews weekly after the trial. Cancel anytime in Google Play.” Footer links: Restore purchases · Terms · Privacy. For users who already used their trial (RevenueCat reports no eligible offer), the card and CTA switch to “<price>/week” / “Subscribe”.
- **Screen 4: Home (App List)** — the core screen.
 - UI Elements: Top App Bar (title “App Cleaner”, Search icon, Sort icon, History icon, Settings gear). Storage card: a segmented bar (Apps / Other / Free) with “96.2 GB of 128 GB used” and “Apps: 21.4 GB”. Tabs: “All apps” · “Unused” (crown) · “Large” (crown). The list: 72dp rows with icon, label, size and one metadata line, trailing checkbox; tapping the row body opens App Details, tapping the checkbox selects. A “Select all” control sits in the list header of each tab. Sticky Selection Bar slides up from the bottom when anything is selected: “3 selected · 1.2 GB” and a Remove Red “Uninstall” button. Adaptive banner ad anchored above the Selection Bar for free users.
 - Sort menu: Name (A–Z), Size (largest first — default), Install date (oldest first), Last updated. Premium adds Last used (least recent first). The chosen sort persists per tab in `DataStore`.
 - Search: expands inline in the top bar, filters by label and package name as you type across the current tab; selection is preserved while filtering (Section 6, item 24).

- Empty & Loading States: grey shimmer rows while the first-ever inventory scan runs. If search matches nothing: “No apps match ‘<query>’”. The All tab is never empty on a real device — if it is, that means `QUERY_ALL_PACKAGES` was stripped (Section 6, item 12).
- **Screen 5: Unused Tab (inside Home)** — threshold chips at the top: 30 · 60 · 90 days (60 default). Rows show “Last opened 4 months ago” or “Not opened since at least Mar 2025”. States, in order: (a) no Usage Access → inline card “See which apps you've stopped using” with “Allow access” → Screen 11; (b) access granted, free user → the count header (“14 apps · 3.4 GB”) is shown in clear, the rows are blurred, and a teal “Unlock with free trial” button opens Screen 3; (c) premium → full list, selectable, with “Select all 14”. Empty (premium): “Nice — every app has been used in the last 60 days.”
- **Screen 6: Large Tab (inside Home)** — apps ranked by total footprint (app + data + cache). Each row shows the total and a thin three-part bar (app / data / cache). Same three access states as Screen 5; the free blurred state shows “Your 10 largest apps use 18.7 GB”.
- **Screen 7: App Details Sheet** — modal bottom sheet from any row. Icon, label, version, “Installed from Google Play” (or “Installed from another source”), install date, last updated. Premium rows: last opened, App / Data / Cache sizes. Free: APK size only, with a small crown line “See data & cache size”. Actions: Open · App info (system settings page, which is also where the user clears cache) · View in Play Store (Play-installed apps only) · Remove Red “Uninstall”.
- **Screen 8: Uninstall Confirm Sheet** — lists the selected apps (icon + label + size), “5 apps · about 2.4 GB”, and one line of expectation-setting: “Android will ask you to confirm each app.” Any flagged app carries a warning chip (e.g. “Your current keyboard”, Section 6 item 10). Buttons: “Cancel” and Remove Red “Uninstall 5 apps”.
- **Screen 9: Uninstall Progress** — full-screen, sits underneath the system dialogs. Determinate bar “Removing 2 of 5”, and the queue as a list with per-app state icons: waiting, in progress, removed (teal check), skipped (grey), failed (red). A “Stop” text button ends the queue after the current dialog.
- **Screen 10: Result Screen** — large count-up “2.4 GB freed”, “5 apps removed”. If any were skipped or failed, a collapsible “2 not removed” section lists them with the reason and a “Try again” action per app. For free users, a native ad slot below the summary, and — only when Usage Access is granted and unused apps exist — a teaser card “You still have 9 apps you haven't opened in 60 days”. Primary CTA “Done”.
- **Screen 11: Usage Access Disclosure** — prominent disclosure required by Play. Illustration, headline “Allow Usage Access”, and three plain lines: what is read (“when each app was last opened, and how much space it uses”), why (“to find unused apps and big apps”), where it goes (“it stays on your phone — never uploaded”). A small animated hint showing the toggle to flip. CTA “Continue to Settings”, secondary “Not now”.
- **Screen 12: Uninstall History** — from the top bar History icon. Removed apps grouped by day, each with its snapshotted icon, label, size and time. Trailing action: “Reinstall” (opens the Play listing) for Play-installed apps; “Not from Play” (disabled) otherwise. Header total: “31 apps removed · 9.8 GB freed”. Overflow: “Clear history”. Empty: “Apps you remove will appear here, so you can always get them back.”
- **Screen 13: Settings** — Language · Theme (System / Light / Dark) · Cleanup reminders (premium toggle + threshold) · Show app size as (APK size / Total) · Manage subscription (deep-link to Play subscriptions) · Restore purchases · Privacy policy · Rate us · Version.

5. Detailed Feature Specifications (Core V1)

Main Feature 1: App Inventory, Search & Sort

- **Core Mechanics & Logic:**
 - Source: `PackageManager.getInstalledPackages(PackageInfoFlags.of(0))` under `QUERY_ALL_PACKAGES`. Keep a package when it is user-installed: `(flags and FLAG_SYSTEM) == 0`. System

apps — including updated system apps — are hidden in V1. The app's own package is always excluded.

- Per-app record: `package_name`, `label`, `version_name`, `first_install_time`, `last_update_time`, `installer` (`getInstallSourceInfo().installingPackageName`), `apk_bytes` and `storage_uuid`. `apk_bytes` is the sum of `File(sourceDir).length()` plus every entry of `splitSourceDirs` — readable without any permission, and it is the free-tier size shown everywhere.
- Cache-first: the whole inventory is persisted in Room. Cold start renders from Room instantly, then a background refresh diffs the live package list against it (added / removed / version changed). Labels are resolved with `loadLabel()` off the main thread; icons load lazily through a Coil fetcher over `loadIcon()` with a memory + disk cache keyed by package + `last_update_time`.
- Live updates: a context-registered receiver for `ACTION_PACKAGE_ADDED`, `ACTION_PACKAGE_REMOVED` and `ACTION_PACKAGE_REPLACED` (data scheme `package`) patches the list while the app is open, so an app removed from the launcher meanwhile disappears without a manual refresh.
- Free storage overview: `StorageStatsManager.getTotalBytes(UUID_DEFAULT)` and `getFreeBytes(UUID_DEFAULT)` — no permission required — drive the storage bar. “Apps” is the sum of `apk_bytes` for free users, and the full app + data + cache total once Usage Access is granted and premium is active. Sizes are formatted with `Formatter.formatShortFileSize()`, which uses the same SI units as system Settings, so our numbers match what the user sees there.
- **Micro-interactions & Edge Cases:** Sort and search are pure in-memory operations over the Room snapshot (300 apps sort in well under 16ms), so they never show a spinner. Selection is a single set of package names shared across tabs, search and sort — it is never silently cleared by switching views. A pull-to-refresh on the list forces a full rescan.

Main Feature 2: Batch Uninstall Engine

- **Core Mechanics & Logic:**
 - Queue: tapping “Uninstall N apps” on the Confirm Sheet writes an `uninstall_queue` batch to Room (`batch_id`, `package`, `order`, `state`, `snapshot_bytes`). Before each item runs, its icon (PNG, 96px), label, installer and size are written to the history snapshot table — text-only when the phone is critically out of space (Section 6, “Storage almost full”). Sizes use the best available measurement: full `StorageStats` when premium and Usage Access are active, `apk_bytes` otherwise — the Result screen says “about” in the free case.
 - Removal: for each item, call `PackageInstaller.uninstall(packageName, statusReceiver.intentSender)`. The receiver gets `STATUS_PENDING_USER_ACTION` first; the engine launches the carried `Intent.EXTRA_INTENT`, which is Android's own confirmation dialog. The final status then arrives on the same receiver: `STATUS_SUCCESS` → removed; `STATUS_FAILURE_ABORTED` → skipped (user tapped Cancel); `STATUS_FAILURE_BLOCKED` → failed, blocked by device policy; anything else → failed with the status code logged (never the package).
 - Verify: on `STATUS_SUCCESS`, confirm `getPackageInfo()` now throws `NameNotFoundException` before marking the item removed, moving its snapshot into History and adding its bytes to the freed total. Only then does the engine start the next item, within 300ms.
 - One dialog at a time, always in the foreground: the engine runs only while Screen 9 is resumed. If the user leaves the app, the queue pauses after the current dialog and resumes when they come back — Android blocks background activity starts, and a dialog appearing over another app would be alarming anyway.
- **Micro-interactions & Edge Cases:** Skipping one app never stops the batch — a Cancel almost always means “not that one”, so the queue moves on and the app lands in “not removed” with a “Try again” action. Three Cancels in a row are different: the engine pauses before the next dialog and asks “Stop removing the rest?” (Section 6, “Several dialogs cancelled in a row”). “Stop” on Screen 9 ends the batch after the current dialog and goes straight to the Result screen with what was done so far. A single-app uninstall (from App Details) uses the

same engine with a queue of one, so History and the freed-space count behave identically. Interruptions, device-admin apps and default-app warnings are detailed in Section 6.

Main Feature 3: Unused Apps Finder & Storage Breakdown (Premium)

- **Core Mechanics & Logic:**

- Access check: `AppOpsManager.unsafeCheckOpNoThrow(OPSTR_GET_USAGE_STATS, uid, packageName) == MODE_ALLOWED`, re-checked in every `onResume`. The grant flow opens `Settings.ACTION_USAGE_ACCESS_SETTINGS` with `Uri.parse("package:" + packageName)` so the user lands on our toggle directly where the OEM supports it. Returning without granting (`onResume`, access still off) puts the user back on the exact tab and scroll position they left, with a gentle inline explanation — never the paywall (Section 6).
- Last used: `UsageStatsManager.queryUsageStats(INTERVAL_YEARLY, now - 2 years, now)`, reduced per package to the latest of `lastTimeUsed` and `lastTimeVisible` (API 29+). Android keeps daily buckets for about a week but yearly buckets for about two years, so the yearly interval is what makes a “90 days” answer possible at all.
- Unused rule: an app is unused at threshold `T` when `now - lastTimeUsed ≥ T` AND `now - first_install_time ≥ T` (an app installed last week is not “unused for 60 days”). The second clause also covers phones restored from a backup, where `first_install_time` is the restore date (Section 6, “Apps restored to a new phone”). An app with no usage record inside the retention window is shown as “Not opened since at least <window start>” — never “Never opened”, which we can’t know.
- Exclusions: never list as unused the current default launcher, keyboard (`Settings.Secure.DEFAULT_INPUT_METHOD`), SMS app and dialer (`Telephony.Sms.getDefaultSmsPackage()`, `TelecomManager.getDefaultDialerPackage()`), any enabled accessibility service or notification listener, or an active device admin. These run constantly without being “opened”, and suggesting their removal would be a bad recommendation dressed up as a smart one.
- Storage breakdown: `StorageStatsManager.queryStatsForPackage(appInfo.storageUuid, packageName, Process.myUserHandle())` → `appBytes`, `dataBytes`, `cacheBytes` (plus `externalCacheBytes` on API 31+). Queried on a background dispatcher in batches of 20 and cached in Room with a timestamp; stale after 24h or on `ACTION_PACKAGE_REPLACED`.
- Gating: both computations run for any user who granted access — that is what makes the count on the blurred teaser real. The entitlement only controls whether rows render in clear and are selectable.
- **Micro-interactions & Edge Cases:** Threshold chips recompute instantly from cached timestamps. The App Details “Clear cache” affordance is an honest deep-link — `Settings.ACTION_APPLICATION_DETAILS_SETTINGS` — labelled “Open App info to clear cache”, never a fake in-app button. Revoked access, OEMs without the package-specific Settings page, and clock changes are covered in Section 6.

Main Feature 4: Uninstall History & Cleanup Reminders

- **Core Mechanics & Logic:**

- History (free): Room table `uninstall_history` (package, label, icon_png, bytes, bytes_is_estimate, installer, version_name, removed_at). Written only from the verified-success path of Feature 2, so it never contains an app that is still installed. “Reinstall” opens `market://details?id=<package>` with a `https://play.google.com/store/apps/details?id=` fallback when no Play Store is present. If the package reappears on the device (reinstalled), its history row gets a “Reinstalled” tag instead of the button.
- Reminders (premium): a `PeriodicWorkRequest` every 7 days (flex 1 day, requires battery not low). The worker checks entitlement and Usage Access, runs the Feature 3 unused rule at the user’s threshold, and posts one notification only if the count grew since the last reminder or 30 days have passed: “9 apps unused

for 60+ days · 2.1 GB. Review them?” The tap deep-links to the Unused tab with those apps pre-selected.

- **Micro-interactions & Edge Cases:** Reminders are off by default until the first successful premium unlock, then on at the Unused tab's threshold, and toggleable in Settings. If notifications are denied, the Settings toggle shows “Allow notifications” linking to `Settings.ACTION_APP_NOTIFICATION_SETTINGS`. History is capped at 1,000 rows (oldest pruned) and excluded from auto-backup, because icons of removed apps have no value on a new device.

6. Error States & Edge Cases

Every failure below has a defined recovery. Uninstalling is irreversible, so the bar here is: the user is never surprised by what was removed, and never told something was removed when it wasn't.

A. Package & Uninstall Integrity

1. **Device-admin apps.** An active device administrator (`DevicePolicyManager.getActiveAdmins()`) cannot be uninstalled until deactivated. Flag it on the Confirm Sheet (“Device admin — deactivate first”), and if it fails in the queue show “Turn off its admin access first” with “Open settings” → `Settings.ACTION_SECURITY_SETTINGS`.
2. **Blocked by policy.** `STATUS_FAILURE_BLOCKED` (work-managed devices, carrier locks, parental controls): mark failed with “Your device doesn't allow removing this app”, no retry button — retrying cannot succeed.
3. **User cancels a dialog.** Skip and continue (Feature 2). The Result screen lists it under “not removed · you cancelled” with “Try again”.
4. **App killed mid-queue.** The queue lives in Room, not memory. On next launch, Home shows a banner “Finish removing 3 apps?” with “Continue” (re-verifies each package still exists first) and “Discard”. Items that were removed before the kill are already in History.
5. **Package changed mid-queue.** Before each item, re-read the package. Already gone (removed elsewhere) → “Already removed”, excluded from the freed total. Updated since selection → re-measure size before the dialog so the snapshot is honest.
6. **Dialog reported success but the package remains.** If `getPackageInfo()` still succeeds after `STATUS_SUCCESS` (seen on some OEM skins that intercept uninstall), mark failed “Android didn't remove this app” with “Try again” — never write it to History or count its bytes.
7. **OEM interception.** Some skins (MIUI/HyperOS, older EMUI) show their own security dialog before the system one. The engine is status-driven, not UI-driven, so extra dialogs only delay the result; no special handling beyond not timing out the queue.
8. **Storage almost full (the usual reason people open the app).** Users typically arrive at 95–99% full, and the app's own writes — the Room snapshot, the history icon PNG, the icon image cache — can fail when internal storage is close to 0 bytes. Graceful fallback, in order: (1) before writing a history icon, check `StorageManager.getAllocatableBytes(UUID_DEFAULT)`; under 5 MB, or on any `IOException/ENOSPC`, skip the PNG and save a text-only snapshot (label, size, installer, removed_at; `icon_png = null`) — History then draws a neutral placeholder icon; (2) the icon image cache never writes to disk under the same threshold (memory only); (3) if even the small Room insert throws `SQLiteFullException`, the uninstall still proceeds — the snapshot is held in memory and written right after the first successful removal, which is exactly when space comes back. A failed snapshot write never blocks, delays or cancels a removal. Logged as `history_snapshot_degraded` (reason: `low_space`, `io_error`, `db_full`).
9. **Very large inventories.** 500+ apps must stay smooth: the list is a `LazyColumn` with stable keys, sizes stream in as they resolve (shimmer per row, not a blocking spinner), and the queue has no batch-size limit.
10. **Default and always-running apps.** Selecting the current keyboard, launcher, SMS app, an enabled accessibility service or an active notification listener adds a warning chip on the Confirm Sheet (“Your current

keyboard — you'll need to pick another”). Warn, don't block: it's the user's phone.

11. **Adopted storage / SD card apps.** Always query sizes with the app's own `storageUuid`, never `UUID_DEFAULT`; if the volume is unmounted, show “Size unavailable” for that app rather than 0 B.
12. **Inventory missing apps.** If the scan returns fewer than 5 user apps on a device reporting many more packages, `QUERY_ALL_PACKAGES` was stripped (sideloaded/modified build). Show an inline card “Some apps may be hidden from App Cleaner” and log `inventory_incomplete`.

B. Permissions, Privacy & Data Accuracy

13. **Usage Access revoked.** Checked on every resume. Unused and Large fall back to the “Allow access” state; cached sizes are cleared so stale numbers are never presented as current; the reminder worker exits silently.
14. **OEM without the package-specific Usage Access page.** If starting the intent with the `package:` URI throws `ActivityNotFoundException` or lands on a list, fall back to the plain `ACTION_USAGE_ACCESS_SETTINGS` and show a 2-second overlay hint: “Find App Cleaner in the list and turn it on.” On recent Android versions (14, 15, 16) several OEMs — Samsung, Xiaomi and Pixel included — accept the `package:` URI but ignore it and open the general list, so the hint must show whenever the list opens, not only on an exception. When the user comes back (`onResume`) without granting, they return to the exact tab and scroll position they left, with a gentle inline card (“Usage access is still off — here's how to turn it on”) and a “Try again” button. Never an aggressive paywall, never a modal, and no repeat of the full disclosure screen.
15. **Apps restored to a new phone.** After Smart Switch or Google Restore, every restored app gets a fresh `first_install_time` (the restore date), and usage history does not transfer. An app the user hasn't opened for 80 days on the old phone therefore can't be called unused until the threshold has passed on the new one. This is deliberate: the rule under-reports right after a restore but never mislabels an app the user may still need. Help copy for an empty Unused list on a recently set-up phone: “Just set up this phone? Unused apps show up after 30 days.”
16. **Clock changed.** Usage timestamps and `System.currentTimeMillis()` can disagree if the user moves the clock. Any `lastTimeUsed` in the future is treated as “used today” — the finder may under-report after a clock change, but never labels an app unused that was just opened. There is no usage counter or free-tier limit anywhere in V1, so there is nothing a clock rewind can unlock.
17. **Inventory in telemetry.** Analytics parameters are validated by a single `AnalyticsEvent` sealed class whose fields are counts, enums and byte buckets only — there is no String field that could carry a package name. Crashlytics custom keys follow the same rule.

C. Monetization, Ads & Offline

18. **Paywall with no connection.** RevenueCat can't fetch offerings: show the benefits, replace the plan card with “Connect to the internet to start your free trial” and a “Try again” button. Never a blank card or an infinite spinner; the close (X) always works.
19. **Premium offline.** RevenueCat's cached `CustomerInfo` keeps `premium` active offline; entitlement is refreshed on the next connected foreground.
20. **Subscription lapses, grace period, account hold.** During Play grace period the entitlement stays active. When it ends, the Unused and Large tabs return to the blurred state, reminders stop, ads return; History, settings and selections are untouched. A one-time Home banner explains: “Your premium ended — renew to keep finding unused apps.”
21. **Trial already used.** Play grants one trial per account; if RevenueCat reports no eligible trial, the paywall copy switches to the plain weekly price (Screen 3). Never show “3 days free” to someone who will be charged today.
22. **Ads never touch the uninstall path.** No interstitial is ever shown between confirmation dialogs or before the Result screen. The only interstitial slot is on leaving the Result screen via “Done”, skipped silently if not loaded within 2 seconds. A banner that fails to fill collapses to zero height.

D. Interaction & Multi-Step Logic

23. **Several dialogs cancelled in a row.** In a batch of 10, 10 system dialogs appear one after another. If the user taps Cancel on 3 consecutive dialogs, the engine does not launch the next one; Screen 9 shows an in-app prompt “Stop removing the rest?” with “Stop” (ends the batch, goes to the Result screen with what was done) and “Keep going” (resets the streak and continues). A single Cancel, or Cancels separated by a removal, never triggers it. Logged as `uninstall_cancel_streak_prompt` (action: stop, keep_going).
24. **Selection across tabs and search.** Selection is one global set. If the search filter hides selected apps, the Selection Bar says “5 selected (2 hidden by search)” and the Confirm Sheet always shows the full list, so nothing invisible is ever uninstalled.
25. **No undo — History is the undo.** Uninstalling can't be reversed in-app. The Confirm Sheet is the last checkpoint, and History with “Reinstall” is the recovery path; app data is not recoverable and the Confirm Sheet does not pretend otherwise.
26. **Rotation and process death on Screen 9.** Queue state is in Room and the engine resumes from the first item not yet in a final state; the pending system dialog is re-requested only if no result came back.
27. **RTL locales.** In Arabic or Hebrew all chrome mirrors — tab order, the checkbox side, the Selection Bar, the progress bar fill direction, back arrows. App labels and icons are user content and render exactly as the app provides them; storage numbers use the locale's digits via `Formatter`.

7. Add to Board (Development Tickets)

- **Ticket 1: [Develop Core] — Inventory, Home & Onboarding.** Target API 36, min 26. Implement the `QUERY_ALL_PACKAGES` inventory with user-app filtering, `apk_bytes` measurement, the Room inventory cache with background diffing and package broadcast receiver, and the Coil icon fetcher. Build Home with the storage card, All / Unused / Large tabs, sort menu, inline search, global selection set and Selection Bar. Build splash, language selector (reusing InkSign's locale module) with dynamic LTR/RTL, the 3-slide carousel, UMP consent and the notification prompt.
- **Ticket 2: [Develop Core] — Batch Uninstall Engine, Result & History.** Build the Room-backed queue, pre-dialog snapshots, `PackageInstaller.uninstall()` with the status receiver, post-success verification, Confirm Sheet (with device-admin and default-app warnings), Progress and Result screens, the resume-after-kill banner, App Details sheet, and Uninstall History with Reinstall. Covers Section 6 items 1–12 and 23–26.
- **Ticket 3: [Develop Premium] — Usage Access, Unused Finder, Storage Breakdown & Reminders.** Build the disclosure screen and grant flow with OEM fallback, the yearly-interval last-used query, the unused rule with exclusions, `StorageStatsManager` batching and caching, the three-state (no access / blurred / premium) tab rendering, the Large tab, and the weekly reminder worker with pre-selected deep link. Covers Section 6 items 13–16.
- **Ticket 4: [Develop Premium] — Subscription, Paywall & Ads.** Integrate Play Billing v8+ and RevenueCat with the weekly base plan and 3-day trial offer, trial-eligibility copy switching, the `premium` entitlement gate, onboarding and contextual paywall triggers, offline and lapse states, and Restore. Wire the banner, Result-screen native ad and exit interstitial with non-blocking no-fill fallback, removed for subscribers. Implement the `AnalyticsEvent` sealed class so no event can carry a package name. Covers Section 6 items 17–22.

8. Basic User Flow

- **Step 1 (Splash & Onboarding):** User opens the app, selects language (UI adapts RTL/LTR), accepts UMP consent, views the 3-slide carousel, allows notifications, and closes or accepts the onboarding paywall -> fires `onboarding_start`, `onboarding_step_viewed`, `onboarding_complete`, `paywall_viewed`.
- **Step 2 (Find):** User lands on Home, already populated and sorted by size, sees “Apps: 21.4 GB” on the storage card, and ticks five apps they recognise as junk -> fires `home_viewed`. (Banner ad above the Selection Bar.)
- **Step 3 (Core Experience — Remove):** User taps “Uninstall”, reviews the Confirm Sheet, confirms five system dialogs in a row and lands on “2.4 GB freed” -> fires `uninstall_batch_started`, `uninstall_batch_completed` (North Star), and `uninstall_failed` for any item that failed.
- **Step 4 (Monetisation):** On the Result screen or Home, the user opens the Unused tab, grants Usage Access via the disclosure screen, sees “14 apps · 3.4 GB” over blurred rows, taps “Unlock with free trial” and starts the 3-day trial -> fires `usage_access_prompt_shown`, `usage_access_granted`, `premium_teaser_viewed`, `paywall_viewed`, `trial_started`.

9. Funnel Analytics Events

No event parameter may contain a package name, app label or free text (Section 2, Section 6 item 17). Byte values are bucketed: <100MB, 100MB–500MB, 500MB–1GB, 1–5GB, >5GB.

- `onboarding_start` — as soon as the language selector appears.
- `onboarding_step_viewed` — params: `step_number` (1, 2, 3), `language_selected`.
- `onboarding_complete` — params: `skipped` (boolean). When the user reaches Home.
- `home_viewed` — params: `app_count`, `storage_used_pct`, `has_usage_access` (boolean), `is_premium` (boolean). Once per session.
- `app_selected` — params: `tab` (all, unused, large), `via` (checkbox, select_all, reminder_preselect). Sampled: first selection per session only.
- `uninstall_batch_started` — params: `app_count`, `bytes_bucket`, `source_tab`, `has_warnings` (boolean).
- `uninstall_batch_completed` — **North Star event**. Params: `removed_count`, `skipped_count`, `failed_count`, `bytes_freed_bucket`, `bytes_is_estimate` (boolean), `seconds_to_complete`, `stopped_early` (boolean), `is_premium`. Fires when the Result screen appears with `removed_count` $\geq$ 1. We optimise the product against weekly users who fire this, and against median seconds from app open to the first one (target under 60).
- `uninstall_failed` — params: `reason` (blocked, device_admin, not_removed_after_success, status_code_other), `status_code`. One per failed item.
- `uninstall_queue_resumed` — params: `remaining_count`, `action` (continue, discard).
- `uninstall_cancel_streak_prompt` — params: `action` (stop, keep_going), `remaining_count`. When three Cancels in a row pause the batch (Section 6).
- `history_snapshot_degraded` — params: `reason` (low_space, io_error, db_full). A history entry was saved without its icon, or late, because the phone was out of space (Section 6).
- `usage_access_prompt_shown` — params: `trigger` (unused_tab, large_tab, storage_card).
- `usage_access_granted` — params: `seconds_in_settings`, `used_fallback_page` (boolean).
- `premium_teaser_viewed` — params: `tab` (unused, large), `found_count`, `bytes_bucket`. The blurred-count state; this is the step that validates decision 3 in Section 0.
- `paywall_viewed` — params: `trigger_source` (onboarding, unused_tab, large_tab, details_sheet, reminder_toggle, settings), `trial_eligible` (boolean), `offering_loaded` (boolean).
- `trial_started` / `subscription_started` — params: `trigger_source`. From the RevenueCat purchase callback.

- `paywall_dismissed` — params: `trigger_source`, `seconds_visible`.
- `history_reinstall_tapped` — params: `days_since_removed_bucket` (0, 1–7, 8–30, >30).
- `reminder_notification_opened` — params: `unused_count`, `threshold_days`.
- `inventory_incomplete` — params: `visible_count`. Detects a stripped `QUERY_ALL_PACKAGES` (Section 6 item 12).

10. Out of Scope for V1

Each of these is a credible V2 and should get its own PRD. None may be retrofitted as a gate on the free select-and-remove flow — that flow staying open and unlimited is what keeps the rating in the 4.5+ band where the category leaders sit.

- **App archiving (Android 15+).** `PackageInstaller.requestArchive()` removes the APK but keeps data and the icon, so the app restores in one tap. The strongest V2 candidate: it is “free up space” with no regret, and no competitor in Appendix A offers it yet. Deferred only because it covers under half the install base today.
- **Accessibility auto-uninstall** (AppDrop’s approach). Play restricts `AccessibilityService` to disability use cases; using it to tap “OK” is a removal risk for the whole listing. Not planned.
- **APK backup / extract before uninstall.** Split APKs mean a backup is a folder of files and restoring needs an installer flow — a medium lift for a rare need.
- **System apps and bloatware** — listing them, “Uninstall updates”, and deep-linking to Disable. Most can’t be removed without root; showing them in V1 adds clutter and “why won’t it uninstall” reviews.
- **Junk / cache cleaning.** Not possible for a normal app since Android 6 without root or Accessibility abuse. Not planned — promising it is what gets cleaner apps suspended (Section 0).
- **Yearly and lifetime plans.** V1 validates the weekly price first; RevenueCat is already in place, so adding base plans later is a configuration change plus a paywall layout, not a re-integration.
- **Work-profile and secondary-user apps.** Visible only to their own profile; would need a device-owner or cross-profile setup.
- **Permission audit / privacy score** (as in Batch Uninstaller & Cleaner). A different product promise (security) with its own review risk; better as its own app.
- **Home screen widget and quick-settings tile** for storage. Nice retention hook, not needed to prove the core loop.

Appendix A: Reference App Findings

Gathered from the live Play Store listings on 28 September 2026.

Reference app	Scale	What to take	What to avoid
Delete apps - Easy Uninstall (Mobique)	1M+ installs 4.4 (33.6K)	The primary reference: one clean screen, multi-select, search and sort, remove in a few taps. Proof the simple select-and-remove loop alone earns a 4.4.	Keyword-stuffed listing (seven feature names that are all the same feature). No sense of what is unused or large, so the user still has to guess.
Uninstaller (SplendApps)	10M+ installs 4.8 (256K)	Highest rating in the category. Sort by name, size and install date in both directions, a storage indicator, and a note up front that system apps can't be removed.	A persistent status-bar notification for quick access — clutter most users switch off.
Easy Uninstaller (Infolife)	10M+ installs 4.5 (181K)	Uninstall history (“recycle bin”), usage tracking, uninstall reminders and a cached app list for instant start — our Features 3 and 4 validated at scale.	An aged feature list (“Support Android 1.6-4.x”) and battery tracking that no longer works on modern Android.
AppDrop: Bulk Uninstaller	50K+ installs 4.2 (591)	Grid view, dark mode, and an honest “no root, no permissions” message.	Accessibility-service auto-uninstall — a Play policy liability — and an APK-extraction premium tier that sells a rare need.
Uninstall Apps – Uninstaller Plus (Sleeptech)	5K+ installs 4.1 (145)	Usage-based “recently used / less used” views and a large-app finder — the exact premium split we chose.	Buries the usage views in a long listing; no evidence shown before asking the user to act.

The gap all of them leave open: none shows the user their own evidence — how many apps they really stopped using and how much space those apps hold — before asking them to do anything. That real count, free to see and premium to act on, is the V1 thesis.

Decision log

- **Name.** “App Cleaner” kept as the brand; the Play title adds “Uninstall Apps” for ranking and to tell it apart from the many near-identical titles. The listing must never claim junk cleaning or speed boosting.
- **Weekly subscription only, with a 3-day trial.** Unused apps finder, per-app storage breakdown, cleanup reminders and no ads are premium; the storage overview, unlimited batch uninstall and History stay free.
- **No connection gate.** Unlike InkSign v1.1, revenue is the subscription, so blocking offline sessions would cost reviews for almost no ad revenue.
- **Usage Access asked for free, before the paywall,** so the paywall can show the user's real numbers.
- **Sequential system dialogs, not Accessibility.** Slower by one tap per app; the listing is safe.
- **v1.1 edge cases.** Added after review: a text-only History fallback when the phone is almost out of space (the removal must never be blocked by our own writes), a “Stop removing the rest?” prompt after three Cancels in a row, a gentle return path when OEMs ignore the Usage Access `package: URI`, and the restored-phone case for the unused rule.